

Guía de Montaje Clase 8: UX Avanzada, Skeleton Screens y Empty States

En esta clase aprenderemos a refinar la interfaz de nuestro proyecto aplicando conceptos de **Experiencia de Usuario (UX)** profesional. Implementaremos **Skeleton Screens** (pantallas esqueleto), **Empty States** (estados vacíos) y transiciones CSS fluidas controladas con JavaScript asíncrono.

Metas de Aprendizaje

1. Comprender el impacto psicológico del tiempo de espera y el concepto de **Latencia Percibida**.
2. Diseñar e implementar **Skeleton Screens** utilizando la animación de pulso nativa de Tailwind CSS (animate-pulse).
3. Crear interfaces de contingencia mediante **Empty States** que guíen de manera proactiva al usuario.
4. Manejar transiciones dinámicas de entrada/salida utilizando manipulación de clases de visibilidad en JavaScript sin refrescar la página.

Guía de Implementación Paso a Paso

Paso 1: Simular la Latencia en Python

Un servidor real tarda tiempo en procesar peticiones complejas o consultar bases de datos. En tu archivo (.py), añadimos `time.sleep(2)` en la ruta `/api/reservas` para congelar artificialmente la respuesta del backend durante 2 segundos. Esto forzará al navegador a mostrar el estado de carga que programaremos a continuación.

Paso 2: Crear el Skeleton Screen con Tailwind CSS

En lugar de un clásico y aburrido spinner de carga, utilizaremos la clase de animación `animate-pulse` de Tailwind sobre bloques grises redondeados que simulan la estructura de las tarjetas reales de reserva.

Paso 3: Diseñar el Estado Vacío (Empty State)

Cuando el usuario borre todas sus reservas, la base de datos quedará limpia. Mostraremos una sección informativa y atractiva que le invite a crear un registro nuevo con un botón directo.

Paso 4: Unificar con JavaScript asíncrono (fetch)

Crearemos una función asíncrona en JS que oculte y muestre las vistas en el momento exacto:

1. Al iniciar la petición, oculta el listado y el estado vacío, y **muestra el Skeleton Screen**.

2. Una vez que llega la promesa resuelta del servidor, **oculta el Skeleton Screen**.
3. Si el JSON está vacío, **muestra el Empty State**. De lo contrario, inyecta las tarjetas reales y las **muestra con una transición de opacidad**.

Guía de Integración: Conectando el Servidor con tus Vistas

Necesitamos realizar algunos ajustes necesarios en algunos archivos existentes y en uno nuevo, iniciaremos con el ejecutable [app.py](#), añadiremos la siguiente librería:

```
import time
```

En la sección de reservas, la ruta que dice:

```
@app.route('/')
def main():
    return render_template('reserva.html')
```

hay que colocar para mejorar la siguiente sección:

```
@app.route('/reserva')
def formulario_reserva():
    """Muestra la vista del formulario de reservas para el QR."""
    return render_template('reserva.html', titulo="Reservas")
```

Sin embargo el archivo principal debe tener su propia configuración nueva, la implementación de nuevas rutas de acceso acoples a un nuevo archivo, de la siguiente forma:

```
# --- ENDPOINTS API AUXILIARES CON LATENCIA (Clase 8) ---
```

```
@app.route('/api/reservas', methods=['GET'])
def api_obtener_reservas():
    if 'user_id' not in session:
        return jsonify({"error": "No autorizado"}), 403
```

```
# Retraso artificial de 2 segundos para forzar que el Dashboard muestre el Skeleton Screen
time.sleep(2)
```

```
if 'mock_reservas' not in session:
```

```

    session['mock_reservas'] = []

    return jsonify(session['mock_reservas'])

@app.route('/api/reservas/crear', methods=['POST'])
def api_crear_reserva_simulada():
    if 'user_id' not in session:
        return jsonify({"error": "No autorizado"}), 403

    # Latencia simulada de escritura de 1.5 segundos
    time.sleep(1.5)

    import random
    servicios = ["Mentoría Python Avanzado 🦩", "Taller de Despliegue Flask 🚀", "Asesoría UI/UX Avanzada 🍕"]
    id_ticket = f"TKT-2026-{random.randint(10, 99)}"

    nueva_reserva = {
        "id": id_ticket,
        "servicio": random.choice(servicios),
        "estado": "Confirmado"
    }

    if 'mock_reservas' not in session:
        session['mock_reservas'] = []

    reservas = session['mock_reservas']
    reservas.append(nueva_reserva)
    session['mock_reservas'] = reservas

    return jsonify(nueva_reserva), 201

@app.route('/api/reservas/limpiar', methods=['POST'])
def api_limpiar_reservas_simuladas():
    if 'user_id' not in session:
        return jsonify({"error": "No autorizado"}), 403

```

```
# Retraso simulado de limpieza de 1 segundo
time.sleep(1)
session['mock_reservas'] = []
return jsonify({"status": "success"}), 200
```

Para que el servidor unificado del **Canvas** funcione correctamente con tu estructura actual en VS Code, debes asegurarte de que cada plantilla HTML reciba los datos adecuados y apunte a las rutas correctas.

A continuación, tienes el código exacto que debes colocar dentro de cada uno de tus archivos existentes en la carpeta templates/.

1. templates/base.html (El Molde de Diseño)

Sustituye el contenido de tu archivo templates/base.html por este código para asegurar la barra de navegación unificada y los estilos CSS con Tailwind:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{{ titulo }} | EcoBooking</title>
  <!-- Tailwind CSS CDN -->
  <script src="https://cdn.tailwindcss.com"></script>
  <!-- Iconos FontAwesome -->
  <link rel="stylesheet"
href="[https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css](https://cdnjs.cloudflare.c
om/ajax/libs/font-awesome/6.4.0/css/all.min.css)">
</head>
<body class="bg-slate-50 font-sans text-slate-800 min-h-screen flex flex-col">

  <!-- Navbar Unificado -->
  <nav class="bg-white border-b border-slate-100 py-4 shadow-sm">
    <div class="max-w-5xl mx-auto px-6 flex justify-between items-center">
      <a href="/" class="text-2xl font-black text-indigo-600 tracking-tight flex items-center gap-2">
        <i class="fa-solid fa-leaf text-emerald-500"></i> EcoBooking
      </a>

      <div class="flex items-center gap-5 font-bold text-sm text-slate-600">

        <a href="/servicios" class="hover:text-indigo-600 transition-colors">
```

Nuestros Servicios

Contáctanos

Catálogo

Reservar QR

{% if 'user_id' in session %}

<a href="/dashboard" class="bg-indigo-50 text-indigo-600 px-4 py-2 rounded-xl
hover:bg-indigo-100 transition-all">

<i class="fa-solid fa-gauge mr-1"></i> Mi Panel

{% else %}

<a href="/login" class="bg-indigo-600 text-white px-4 py-2 rounded-xl hover:bg-indigo-700
transition-all">

Iniciar Sesión

{% endif %}

</div>

</div>

</nav>

<!-- Contenido Principal -->

<main class="flex-grow">

<!-- Mostrar Mensajes Flash de Flask de forma Global -->

<div class="max-w-4xl mx-auto px-6 mt-6">

{% with messages = get_flashed_messages(with_categories=true) %}

{% if messages %}

<div class="space-y-2">

{% for category, message in messages %}

<div class="p-4 rounded-xl text-sm font-bold text-center border

{% if category == 'success' %} bg-emerald-50 text-emerald-700 border-emerald-200

{% elif category == 'warning' %} bg-amber-50 text-amber-700 border-amber-200

{% else %} bg-rose-50 text-rose-700 border-rose-200 {% endif %}">

{{ message }}

</div>

{% endfor %}

</div>

{% endif %}

{% endwith %}

</div>

```

    {% block content %}
    {% endblock %}
</main>

<!-- Pie de Página -->
<footer class="bg-slate-950 text-slate-500 text-center py-6 border-t border-slate-900 mt-12">
    <p class="text-xs">© 2026 - EcoBooking | Solución Tecnológica Integrada</p>
</footer>

</body>
</html>

```

2. templates/index.html (El Buscador en Tiempo Real - Clase 5)

Sustituye tu templates/index.html para unificar el catálogo CRUD de productos de la Clase 4 junto con el buscador dinámico por API de la Clase 5:

```

{% extends "base.html" %}

{% block content %}
<div class="max-w-4xl mx-auto px-6 py-10">
    <div class="text-center mb-10">
        <h1 class="text-4xl font-black text-slate-900 tracking-tight">Catálogo de Productos</h1>
        <p class="text-slate-500 mt-2">Agrega productos locales a SQLite y búscalos asincrónamente en
tiempo real.</p>
    </div>

    <!-- Buscador en tiempo real (Fetch API - Clase 5) -->
    <div class="bg-white p-6 rounded-3xl border border-slate-100 shadow-xl mb-10">
        <h3 class="font-bold text-slate-800 mb-3 flex items-center gap-2">
            <i class="fa-solid fa-magnifying-glass text-indigo-500"></i> Buscador Predictivo (Fetch API)
        </h3>
        <input type="text" id="inputBusqueda" placeholder="Escribe el nombre de un producto para
buscar..."
            class="w-full p-4 rounded-xl border border-slate-200 focus:outline-none focus:ring-2
focus:ring-indigo-500 transition-all text-sm">

        <!-- Contenedor dinámico de resultados asíncronos -->
        <div id="resultados-busqueda" class="mt-4 hidden space-y-2"></div>
    </div>

    <!-- Formulario para agregar productos (CRUD - Clase 4) -->
    <div class="bg-white p-8 rounded-3xl border border-slate-100 shadow-xl mb-10">
        <h3 class="font-bold text-lg text-slate-800 mb-4">Registrar Nuevo Producto en SQLite</h3>
        <form action="/agregar" method="POST" class="grid grid-cols-1 md:grid-cols-3 gap-4">
            <input type="text" name="nombre" placeholder="Nombre (ej. Champú Orgánico)" required

```

```

        class="p-3.5 rounded-xl border border-slate-200 outline-none focus:ring-2
focus:ring-indigo-500 text-sm">
        <input type="number" step="0.01" name="precio" placeholder="Precio ($)" required
        class="p-3.5 rounded-xl border border-slate-200 outline-none focus:ring-2
focus:ring-indigo-500 text-sm">
        <button type="submit" class="bg-indigo-600 hover:bg-indigo-700 text-white font-bold py-3.5
rounded-xl text-sm transition-all shadow-md shadow-indigo-100">
        Guardar en DB
    </button>
</form>
</div>

```

```

<!-- Tabla de lectura de la base de datos (CRUD) -->
<div class="bg-white rounded-3xl border border-slate-100 shadow-xl overflow-hidden">
    <table class="w-full text-left">
        <thead class="bg-slate-900 text-slate-200 text-xs uppercase font-black tracking-wider">
            <tr>
                <th class="p-4 pl-6">ID</th>
                <th class="p-4">Producto</th>
                <th class="p-4">Precio</th>
                <th class="p-4 text-center">Acciones</th>
            </tr>
        </thead>
        <tbody class="divide-y divide-slate-100 text-sm">
            {% for p in productos %}
            <tr class="hover:bg-slate-50/50 transition-colors">
                <td class="p-4 pl-6 font-mono text-slate-400">#{{ p.id }}</td>
                <td class="p-4 font-bold text-slate-700">{{ p.nombre }}</td>
                <td class="p-4 font-black text-indigo-600">${{ "%.2f"|format(p.precio) }}</td>
                <td class="p-4 text-center">
                    <a href="/borrar/{{ p.id }}" class="bg-rose-50 hover:bg-rose-100 text-rose-600 font-bold
px-3 py-1.5 rounded-lg text-xs transition-colors">
                        Eliminar
                    </a>
                </td>
            </tr>
            {% else %}
            <tr>
                <td colspan="4" class="p-10 text-center text-slate-400">
                    La base de datos está vacía. ¡Agrega tu primer producto arriba!
                </td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
</div>
</div>

```

```

<!-- Script del Buscador en tiempo real -->
<script>
  const input = document.getElementById('inputBusqueda');
  const resultadosDiv = document.getElementById('resultados-busqueda');

  input.addEventListener('input', async (e) => {
    const query = e.target.value.trim();

    if (query.length < 2) {
      resultadosDiv.innerHTML = "";
      resultadosDiv.classList.add('hidden');
      return;
    }

    try {
      const res = await fetch(`/api/buscar?q=${query}`);
      const productos = await res.json();

      resultadosDiv.innerHTML = "";
      resultadosDiv.classList.remove('hidden');

      if (productos.length === 0) {
        resultadosDiv.innerHTML = `<p class="text-xs text-slate-400 italic p-2">No se encontraron
productos coincidentes.</p>`;
      } else {
        productos.forEach(p => {
          resultadosDiv.innerHTML += `
            <div class="flex justify-between items-center p-3 bg-indigo-50/30 border
border-indigo-100/50 rounded-xl text-xs">
              <span class="font-bold text-slate-700">${p.nombre}</span>
              <span class="font-black text-indigo-600">${p.precio.toFixed(2)}</span>
            </div>
          `;
        });
      }
    } catch (err) {
      console.error("Error al buscar:", err);
    }
  });
</script>
{% endblock %}

```

3. templates/reserva.html (El Formulario de WhatsApp y QR - Clase 7)

Sustituye tu templates/reserva.html para asegurarte de que el formulario de envío coincida con las variables esperadas en app.py:


```
{% extends "base.html" %}
```

```
{% block content %}
```

```
<div class="min-h-[75vh] flex items-center justify-center py-12 px-6">
```

```
<div class="max-w-md w-full space-y-8 bg-white p-10 rounded-3xl shadow-2xl border
border-slate-100">
```

```
<div class="text-center">
```

```
<div class="mx-auto h-16 w-16 rounded-2xl bg-emerald-50 text-emerald-600 flex items-center
justify-center text-2xl mb-4">
```

```
<i class="fa-solid fa-ticket"></i>
```

```
</div>
```

```
<h2 class="text-3xl font-black text-slate-900 tracking-tight">Reservar Cita</h2>
```

```
<p class="mt-2 text-sm text-slate-500">
```

```
Genera tu código QR en el servidor y recíbelo por WhatsApp usando Twilio.
```

```
</p>
```

```
</div>
```

```
<form class="space-y-6" action="/reservar" method="POST">
```

```
<div class="space-y-4">
```

```
<div>
```

```
<label for="nombre" class="block text-sm font-bold text-slate-700 mb-1">Nombre
Completo</label>
```

```
<input id="nombre" name="nombre" type="text" required
```

```
class="w-full px-4 py-3 border border-slate-200 rounded-xl focus:outline-none
focus:ring-2 focus:ring-indigo-500 text-sm"
```

```
placeholder="Ej: Claudia Natalia">
```

```
</div>
```

```
<div>
```

```
<label for="telefono" class="block text-sm font-bold text-slate-700 mb-1">Número de
WhatsApp</label>
```

```
<input id="telefono" name="telefono" type="tel" required
```

```
class="w-full px-4 py-3 border border-slate-200 rounded-xl focus:outline-none
focus:ring-2 focus:ring-indigo-500 text-sm"
```

```
placeholder="Ej: +573001234567">
```

```
<p class="mt-1.5 text-[11px] text-slate-400 leading-snug">
```

```
⚠ Obligatorio incluir el prefijo de país con el signo <b>+</b> (ej: +57 para Colombia).
```

```
</p>
```

```
</div>
```

```
<div>
```

```
<label for="servicio" class="block text-sm font-bold text-slate-700 mb-1">Servicio a
Reservar</label>
```

```
<select id="servicio" name="servicio" required
```

```
class="block w-full px-4 py-3 border border-slate-200 bg-white rounded-xl
text-slate-700 focus:outline-none focus:ring-2 focus:ring-indigo-500 text-sm">
```

```
<option value="" disabled selected>Selecciona una opción...</option>
```

```

        <option value="Mentoría Python y APIs">Mentoría Python y APIs 🐍 </option>
        <option value="Taller de Despliegue Express">Taller de Despliegue Express 🚀 </option>
        <option value="Asesoría UI/UX Avanzada">Asesoría UI/UX Avanzada 🧑‍🎓 </option>
    </select>
</div>
</div>

    <button type="submit"
        class="w-full flex justify-center items-center gap-2 py-4 px-4 border border-transparent
text-sm font-black rounded-xl text-white bg-emerald-600 hover:bg-emerald-700 transition-all
shadow-lg shadow-emerald-100">
        <i class="fa-brands fa-whatsapp text-lg"></i>
        Confirmar Reserva y Enviar QR
    </button>
</form>

</div>
</div>
{% endblock %}

```

4. templates/dashboard.html (El Panel de Control de Estados - Clase 8)

Sustituye por completo el archivo templates/dashboard.html por este código para activar el panel dinámico con los tres estados asíncronos y animaciones fluidas (Skeleton, Empty State e inyección de datos reales):

```

{% extends "base.html" %}

{% block content %}
<!-- Animación de entrada progresiva de elementos reales -->
<style>
    .card-entry {
        animation: cardSlideUp 0.5s ease-out forwards;
    }
    @keyframes cardSlideUp {
        from {
            opacity: 0;
            transform: translateY(12px);
        }
        to {
            opacity: 1;
            transform: translateY(0);
        }
    }
</style>

```

```

<div class="max-w-5xl mx-auto py-12 px-6">
  <div class="bg-white rounded-3xl shadow-xl border border-slate-100 overflow-hidden">

    <!-- Encabezado de Sesión -->
    <div class="bg-indigo-600 p-8 text-white flex flex-col md:flex-row items-center justify-between gap-6">
      <div class="flex items-center gap-6">
        <div class="w-20 h-20 bg-indigo-400 rounded-full flex items-center justify-center text-3xl font-bold border-4 border-indigo-300">
          {{ nombre[0] | upper }}
        </div>
        <div>
          <h1 class="text-3xl font-black">Panel de {{ nombre }}</h1>
          <p class="text-indigo-100">Sesión iniciada correctamente  </p>
        </div>
      </div>
      <a href="/logout" class="bg-white/10 hover:bg-white/20 px-6 py-2.5 rounded-xl font-bold transition-all backdrop-blur-md border border-white/20 text-sm">
        Cerrar Sesión
      </a>
    </div>

    <!-- Acciones del Sandbox de Experiencia de Usuario (Clase 8) -->
    <div class="p-8 border-b border-slate-50 bg-slate-50 flex flex-col sm:flex-row justify-between items-center gap-4">
      <div>
        <h3 class="font-bold text-slate-800">Evaluación de Experiencia de Usuario</h3>
        <p class="text-xs text-slate-400">Prueba cómo responde la UI ante retrasos artificiales del servidor.</p>
      </div>
      <div class="flex gap-3 w-full sm:w-auto">
        <button id="btn-crear-mock" class="flex-grow sm:flex-grow-0 bg-indigo-600 hover:bg-indigo-700 text-white font-bold px-4 py-2.5 rounded-xl text-xs transition-all flex items-center justify-center gap-2">
          <i class="fa-solid fa-plus"></i> Nueva Reserva (1.5s delay)
        </button>
        <button id="btn-vaciar-mock" class="flex-grow sm:flex-grow-0 bg-rose-50 hover:bg-rose-100 text-rose-600 font-bold px-4 py-2.5 rounded-xl text-xs transition-all flex items-center justify-center gap-2">
          <i class="fa-solid fa-trash-can"></i> Limpiar Agenda (1s delay)
        </button>
      </div>
    </div>

    <!-- ÁREA DINÁMICA DE ESTADOS ASÍNCRONOS -->
    <div class="p-8">

```

```

<!-- ===== -->
<!-- ESTADO 1: SKELETON SCREEN (Pantalla de carga estructural) -->
<!-- ===== -->
<div id="loading-state" class="space-y-4">
  <div class="bg-white rounded-2xl p-5 border border-slate-100 flex items-center
justify-between animate-pulse">
    <div class="flex items-center gap-4 w-2/3">
      <div class="w-12 h-12 bg-slate-200 rounded-xl"></div>
      <div class="flex-grow space-y-2">
        <div class="h-4 bg-slate-200 rounded w-1/3"></div>
        <div class="h-3 bg-slate-200 rounded w-1/2"></div>
      </div>
    </div>
    <div class="w-24 h-8 bg-slate-200 rounded-lg"></div>
  </div>
  <div class="bg-white rounded-2xl p-5 border border-slate-100 flex items-center
justify-between animate-pulse">
    <div class="flex items-center gap-4 w-2/3">
      <div class="w-12 h-12 bg-slate-200 rounded-xl"></div>
      <div class="flex-grow space-y-2">
        <div class="h-4 bg-slate-200 rounded w-1/4"></div>
        <div class="h-3 bg-slate-200 rounded w-1/3"></div>
      </div>
    </div>
    <div class="w-24 h-8 bg-slate-200 rounded-lg"></div>
  </div>
</div>

<!-- ===== -->
<!-- ESTADO 2: EMPTY STATE (Pantalla activa si no hay reservas) -->
<!-- ===== -->
<div id="empty-state" class="hidden text-center py-12 max-w-sm mx-auto card-entry">
  <div class="w-16 h-16 bg-indigo-50 text-indigo-500 rounded-2xl flex items-center justify-center
mx-auto mb-4">
    <i class="fa-solid fa-calendar-xmark text-2xl"></i>
  </div>
  <h3 class="text-lg font-bold text-slate-800">No hay reservas registradas</h3>
  <p class="text-slate-400 mt-1 text-xs leading-relaxed">
    Tu agenda se encuentra actualmente vacía. Haz clic en "Nueva Reserva" para simular un
proceso de guardado asíncrono.
  </p>
</div>

<!-- ===== -->
<!-- ESTADO 3: CONTENEDOR DE DATOS REALES -->
<!-- ===== -->
<div id="data-state" class="hidden space-y-4">
  <!-- Se inyectarán de manera dinámica vía JavaScript -->

```

```

    </div>

</div>

</div>
</div>

<!-- SCRIPT DE CONTROL DE INTERFAZ -->
<script>
    const loader = document.getElementById('loading-state');
    const emptyState = document.getElementById('empty-state');
    const dataState = document.getElementById('data-state');

    const btnCrear = document.getElementById('btn-crear-mock');
    const btnVaciar = document.getElementById('btn-vaciar-mock');

    // Función asíncrona que hace fetch a nuestra API con delay de red de prueba
    async function cargarReservas() {
        // Encendemos el Skeleton de carga y apagamos estados secundarios
        loader.classList.remove('hidden');
        emptyState.classList.add('hidden');
        dataState.classList.add('hidden');

        try {
            const response = await fetch('/api/reservas');
            if (!response.ok) throw new Error("Error al obtener los datos de la sesión.");

            const reservas = await response.json();
            dataState.innerHTML = "";

            if (reservas.length === 0) {
                // Si la DB simulada está vacía, mostramos el Empty State
                emptyState.classList.remove('hidden');
            } else {
                // Inyectamos cada tarjeta con su animación suave 'card-entry'
                reservas.forEach(r => {
                    const tarjeta = `
                        <div class="bg-white rounded-2xl p-5 border border-slate-100 shadow-sm flex
items-center justify-between hover:shadow-md transition-all duration-300 card-entry">
                            <div class="flex items-center gap-4">
                                <div class="w-12 h-12 bg-indigo-50 text-indigo-600 rounded-xl flex items-center
justify-center text-xl">
                                    
                                </div>
                                <div>
                                    <h4 class="font-bold text-slate-800">${r.servicio}</h4>
                                    <p class="text-slate-400 text-xs flex items-center gap-1 mt-0.5">
                                        <i class="fa-regular fa-id-badge"></i> Código: ${r.id}

```

```

        </p>
      </div>
    </div>
    <span class="bg-emerald-50 text-emerald-700 text-xs font-black px-3 py-1.5 rounded-lg uppercase tracking-wider">
      ${r.estado}
    </span>
  </div>
  `;
  dataState.innerHTML += tarjeta;
});
dataState.classList.remove('hidden');
}
} catch (error) {
  console.error(error);
} finally {
  // El esqueleto de carga se oculta de forma obligatoria al resolver la promesa
  loader.classList.add('hidden');
}
}
}

```

// Evento para simular la creación de una reserva (POST)

btnCrear.addEventListener('click', async () => {

btnCrear.disabled = true;

btnCrear.innerHTML = `<i class="fa-solid fa-spinner animate-spin"></i> Guardando...`;

try {

const response = await fetch('/api/reservas/crear', {

method: 'POST',

headers: { 'Content-Type': 'application/json' }

});

if (response.ok) {

await cargarReservas(); // Recarga la lista para ejecutar el flujo completo

}

} catch (e) {

console.error(e);

} finally {

btnCrear.disabled = false;

btnCrear.innerHTML = `<i class="fa-solid fa-plus text-xs"></i> Nueva Reserva (1.5s delay)`;

}

});

// Evento para limpiar la agenda de reservas (POST)

btnVaciar.addEventListener('click', async () => {

if (!confirm("¿Deseas vaciar la agenda para evaluar el Empty State?")) return;

btnVaciar.disabled = true;

btnVaciar.innerHTML = `<i class="fa-solid fa-spinner animate-spin"></i> Limpiando...`;

```

    try {
      const response = await fetch('/api/reservas/limpiar', { method: 'POST' });
      if (response.ok) {
        await cargarReservas();
      }
    } catch (e) {
      console.error(e);
    } finally {
      btnVaciar.disabled = false;
      btnVaciar.innerHTML = `<i class="fa-solid fa-trash-can text-xs"></i> Limpiar Agenda (1s delay)`;
    }
  });

  // Lanzar consulta asíncrona tan pronto se cargue la estructura DOM
  window.addEventListener('DOMContentLoaded', cargarReservas);
</script>
{% endblock %}

```

5. templates/reservas_lista.html (El Panel de Control de Estados - Clase 8 complemento)

```

{% extends "base.html" %}

{% block content %}
<div class="max-w-4xl mx-auto px-4 py-12">

  <div class="flex justify-between items-center mb-8">
    <div>
      <h1 class="text-3xl font-black text-slate-900 tracking-tight">Mis Reservas</h1>
      <p class="text-slate-500 text-sm">Administra tus próximas mentorías y talleres.</p>
    </div>
    <button id="btn-cargar" class="bg-indigo-600 hover:bg-indigo-700 text-white font-bold px-6 py-3 rounded-xl shadow-lg shadow-indigo-100 transition-all flex items-center gap-2">
      <i class="fa-solid fa-rotate text-sm"></i> Simular Carga de Servidor
    </button>
  </div>

  <div id="estado-vacio" class="bg-white rounded-3xl border border-slate-100 p-16 text-center shadow-xl">
    <div class="w-20 h-20 bg-indigo-50 text-indigo-500 rounded-full flex items-center justify-center mx-auto mb-6">

```

```

        <i class="fa-solid fa-calendar-xmark text-3xl"></i>
    </div>

    <h3 class="text-xl font-bold text-slate-800">No hay reservas registradas</h3>

    <p class="text-slate-400 mt-2 max-w-sm mx-auto text-sm">
        Aún no has solicitado ninguna mentoría. Presiona el botón de arriba para buscar respuestas del
        servidor.
    </p>
</div>

```

```

<div id="estado-cargando" class="hidden space-y-4">
    <div class="bg-white rounded-2xl p-6 border border-slate-100 shadow-md flex items-center
justify-between animate-pulse">
        <div class="flex items-center gap-4 w-2/3">
            <div class="w-12 h-12 bg-slate-200 rounded-xl"></div>
            <div class="flex-1 space-y-2">
                <div class="h-4 bg-slate-200 rounded w-1/3"></div>
                <div class="h-3 bg-slate-200 rounded w-1/2"></div>
            </div>
        </div>
        <div class="w-24 h-8 bg-slate-200 rounded-lg"></div>
    </div>

    <div class="bg-white rounded-2xl p-6 border border-slate-100 shadow-md flex items-center
justify-between animate-pulse">
        <div class="flex items-center gap-4 w-2/3">
            <div class="w-12 h-12 bg-slate-200 rounded-xl"></div>
            <div class="flex-1 space-y-2">
                <div class="h-4 bg-slate-200 rounded w-1/4"></div>
                <div class="h-3 bg-slate-200 rounded w-1/3"></div>
            </div>
        </div>
        <div class="w-24 h-8 bg-slate-200 rounded-lg"></div>
    </div>
</div>

```

```

<div id="estado-datos" class="hidden space-y-4">
    <div class="bg-white rounded-2xl p-6 border border-slate-100 shadow-xl flex items-center
justify-between hover:scale-[1.01] transition-all duration-300">

```



```

<div class="flex items-center gap-4">
  <div class="w-12 h-12 bg-emerald-50 text-emerald-600 rounded-xl flex items-center
justify-center text-xl font-bold">
    🦧
  </div>
  <div>
    <h4 class="font-bold text-slate-800 text-lg">Mentoría Python Avanzado</h4>
    <p class="text-slate-400 text-sm flex items-center gap-1">
      <i class="fa-regular fa-clock"></i> Mañana a las 4:00 PM
    </p>
  </div>
</div>

<span class="bg-emerald-50 text-emerald-700 text-xs font-black px-3 py-1.5 rounded-full
uppercase tracking-wider">
  Confirmado
</span>
</div>

<div class="bg-white rounded-2xl p-6 border border-slate-100 shadow-xl flex items-center
justify-between hover:scale-[1.01] transition-all duration-300">
  <div class="flex items-center gap-4">
    <div class="w-12 h-12 bg-indigo-50 text-indigo-600 rounded-xl flex items-center justify-center
text-xl font-bold">
      🚀
    </div>
    <div>
      <h4 class="font-bold text-slate-800 text-lg">Taller de Despliegue Flask</h4>
      <p class="text-slate-400 text-sm flex items-center gap-1">
        <i class="fa-regular fa-clock"></i> Viernes, 05 de Junio
      </p>
    </div>
  </div>
  <span class="bg-amber-50 text-amber-700 text-xs font-black px-3 py-1.5 rounded-full uppercase
tracking-wider">
    Pendiente
  </span>
</div>

```

```
</div>
```

```
</div>
```

```
<script>
```

```
const btnCargar = document.getElementById('btn-cargar');  
const estadoVacio = document.getElementById('estado-vacio');  
const estadoCargando = document.getElementById('estado-cargando');  
const estadoDatos = document.getElementById('estado-datos');
```

```
btnCargar.addEventListener('click', () => {  
  // 1. Ocultamos el estado vacio inmediatamente  
  estadoVacio.classList.add('hidden');  
  
  // 2. Mostramos la pantalla esqueleto (Skeleton Screen)  
  estadoCargando.classList.remove('hidden');  
  
  // 3. Simulamos una espera de servidor de 2.5 segundos (2500ms)  
  setTimeout(() => {  
    // Ocultamos la animación de carga  
    estadoCargando.classList.add('hidden');  
    // Mostramos los datos reales con una transición suave  
    estadoDatos.classList.remove('hidden');  
  }, 2500);  
});
```

```
</script>
```

```
{% endblock %}
```

1. La Ilusión de la Velocidad

Cuando un backend consulta bases de datos grandes, procesa archivos o consume APIs externas, la respuesta no es inmediata. Si la pantalla se queda congelada, el usuario asume que el sistema falló.

- **El Spinner (Cargador tradicional):** Concentra la atención en la espera. El usuario ve girar el círculo y siente que el tiempo pasa más lento.
- **El Skeleton Screen:** Imita la silueta de lo que va a cargar. El cerebro del usuario se

enfoca en el contenido que viene, reduciendo la ansiedad y la tasa de rebote en un 40%.

2. Anatomía de un Empty State Exitoso

Nunca dejes una pantalla vacía en blanco. Un estado vacío profesional siempre debe contener:

- **Iconografía o Ilustración:** Que represente visualmente la ausencia de datos sin ser punitivo.
- **Título Claro:** "No tienes reservas registradas".
- **Texto de Apoyo:** "Tus citas agendadas aparecerán en esta sección".
- **Llamado a la Acción (CTA):** Un botón claro de "Agendar mi primera cita".



Checklist de Errores Comunes de los Estudiantes

- **¿Cómo funciona el parpadeo gris del Skeleton?**
 - *Explicación:* Tailwind tiene una clase nativa de animación llamada `animate-pulse`. Al ponerla en el contenedor gris, genera automáticamente ese efecto de desvanecimiento suave de opacidad.
- **Error: Al dar clic, el botón no hace nada**
- *Causa:* Los estudiantes suelen equivocarse al escribir los nombres de las variables en JavaScript o se les olvida poner las IDs correctas en los divs (`id="estado-vacio"`, etc.).
- *Solución:* Asegúrate de revisar que coincidan exactamente los textos de `document.getElementById` con los atributos `id` del HTML.
- **Error: Las animaciones de Tailwind no se activan.**
 - *Causa:* El CDN de Tailwind no se cargó correctamente o hay errores de sintaxis en las clases (ej: escribir `animation-pulse` en lugar de `animate-pulse`).
 - *Solución:* Asegurarse de tener la etiqueta de script del CDN de Tailwind en la cabecera.
- **Error: El esqueleto no desaparece y se sobrepone al contenido real.**
 - *Causa:* El código de JavaScript no ejecutó el método `.classList.add('hidden')` sobre el contenedor del esqueleto una vez resuelta la promesa.
 - *Solución:* Comprobar que el bloque `finally {}` de la petición asíncrona apague el estado de carga de forma obligatoria.
- **Error de llamadas de origen cruzado o rutas inexistentes:**
 - *Causa:* El `fetch('/api/reservas')` apunta a una ruta mal escrita en Python o a un puerto diferente.
 - *Solución:* Validar que el endpoint en `@app.route` de Flask coincida letra por letra con la URL invocada en JavaScript.
- **Error: Al limpiar la tabla, la pantalla se queda congelada en gris.**
 - *Causa:* El bloque `finally {}` de JavaScript falló o el desarrollador olvidó ocultar el cargador (`loader.classList.add('hidden')`).
 - *Solución:* Asegurarse de que el esqueleto se oculte obligatoriamente al finalizar la petición, sin importar si tuvo éxito o falló.

- *Causa:* Se escribió mal la clase de Tailwind (ej: animation-pulse en lugar de animate-pulse).
- *Solución:* Validar la sintaxis en la documentación oficial de Tailwind CSS.